

# Pramo Jewels

## Coding Standards & Naming Conventions

Document ID: PJ-CODE-001

Version: 1.0

Status: Approved for Development

| Field    | Value                                                                           |
|----------|---------------------------------------------------------------------------------|
| Project  | Pramo Jewels Premium E-Commerce Platform                                        |
| Purpose  | Establish consistent coding practices and naming conventions across the project |
| Audience | Frontend Developers, Backend Developers, QA, DevOps, AI Coding Agents           |
| Version  | 1.0                                                                             |

### 1. General Principles

Write clean, readable, maintainable code following SOLID, DRY and KISS principles. Prefer composition over duplication and keep functions focused on a single responsibility.

### 2. Naming Conventions

Variables: camelCase  
Functions: camelCase verbs (getProducts, createOrder)  
React Components: PascalCase  
Classes: PascalCase  
Interfaces/Types: PascalCase  
Constants: UPPER\_SNAKE\_CASE  
Database tables: snake\_case plural  
Database columns: snake\_case  
API endpoints: kebab-case

### 3. File & Folder Naming

React components: ProductCard.tsx  
Hooks: useCart.ts  
Utilities: formatCurrency.ts  
Laravel models: Product.php  
Controllers: ProductController.php  
Services: OrderService.php

### 4. Code Formatting

Use Prettier for frontend formatting and Laravel Pint for backend formatting. Enforce formatting through pre-commit hooks and CI.

### 5. Documentation

Document public classes, reusable utilities, complex business logic and APIs. Keep README files updated for major modules.

## 6. Error Handling

Return consistent error responses, avoid exposing sensitive information, log exceptions centrally and provide meaningful messages.

## 7. Security Practices

Validate and sanitize all inputs, use parameterized queries/ORM, never hardcode secrets, follow least-privilege access and secure dependency management.

## 8. Git Standards

Feature branches: feature/

Bug fixes: fix/

Hotfixes: hotfix/

Commit messages follow Conventional Commits (feat:, fix:, docs:, refactor:, test:, chore:).

## 9. Testing Standards

Write unit tests for business logic, integration tests for APIs and end-to-end tests for critical customer journeys. New features should include corresponding tests.

## 10. Code Review Checklist

- Naming consistency
- Readability
- Performance
- Security
- Accessibility
- Test coverage
- Documentation updated

## 11. Quality Gates

All pull requests must pass linting, formatting, static analysis, automated tests and peer review before merging.

## 12. Governance

These standards apply to all contributors. Any exceptions require architectural approval and must be documented.